Lyumiao Zhang

26WN-CS630-154W - Database Systems

Final project

Project topic: **Restaurant Order Management**

## Summary

In this project, I built a restaurant order management system using a relational database. The goal was to design a database that can store customers, restaurants, menu items, and orders in a structured way. I also made sure the database follows normalization rules (up to 3NF) to avoid redundancy and keep the data consistent

## Introduction

The purpose of this project is to design and implement a database that supports how a restaurant handles orders. Customers can place orders, restaurants have menus, and each order can contain multiple items.

This project helped me understand how to organize data properly, use relationships between tables, and apply SQL to create and manage a database.

## Data Modeling

The system consists of five main entities:

1. **CUSTOMER:** stores customer information such as name, email, and phone number
2. **RESTAURANT:** stores restaurant details including name and location
3. **MENU_ITEM:** represents items offered by a restaurant. Each menu item belongs to one restaurant
4. **ORDERS:** stores order-level information such as the customer, restaurant, order date, and status
5. **ORDER_ITEM:** represents individual items within an order. This table resolves the many-to-many relationship between orders and menu items

## Relationships

The relationships in the database are:

- One customer can place many orders
- One restaurant can have many menu items
- One order can have many order items
- One menu item can appear in many order items

## Normalization (3NF)

The database is normalized up to Third Normal Form:

- **1NF:** All fields contain atomic values (no repeating groups)
- **2NF:** All non-key attributes depend on the whole primary key
- **3NF:** There are no transitive dependencies

The use of the ORDER_ITEM table helps remove the many-to-many relationship between orders and menu items, which keeps the design clean and avoids duplicate data

**Design Decisions**

One important decision was to create the ORDER_ITEM table instead of storing items directly in the ORDERS table. This allows each order to have multiple items and avoids repeating data. I also used foreign keys to make sure relationships between tables are enforced. For example, each order must be linked to a valid customer and restaurant. I decided to keep the total_amount column in the ORDERS table. Even though it could be calculated from ORDER_ITEM, storing it makes it easier and faster to query total order values.

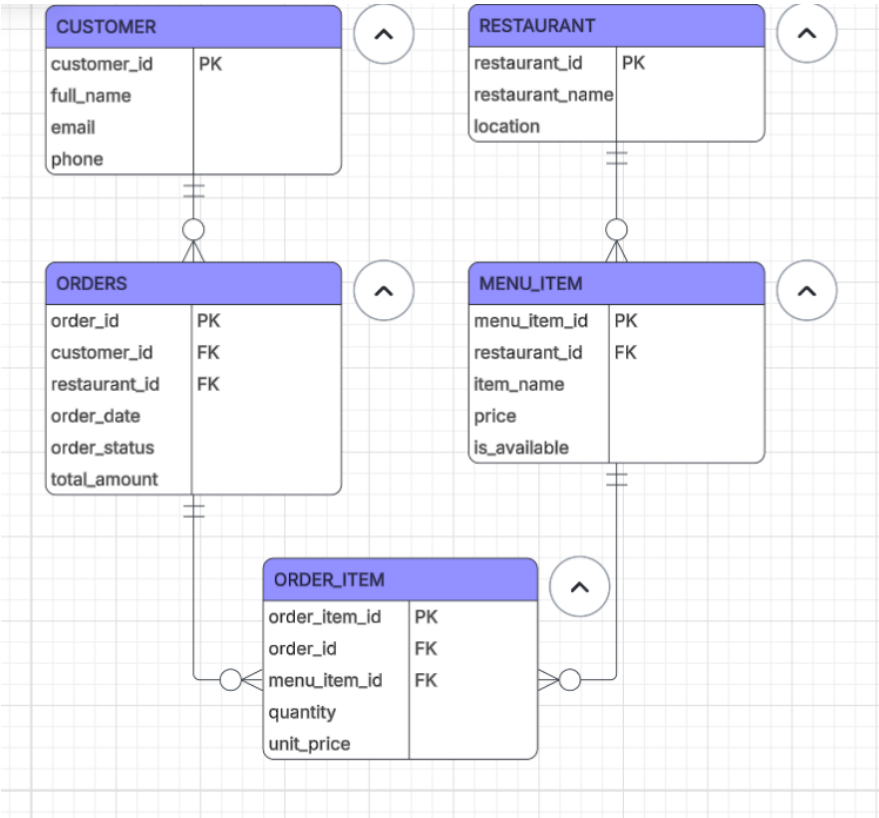
**SQL Implementation**

The database was implemented using Oracle SQL. I created three main SQL files:

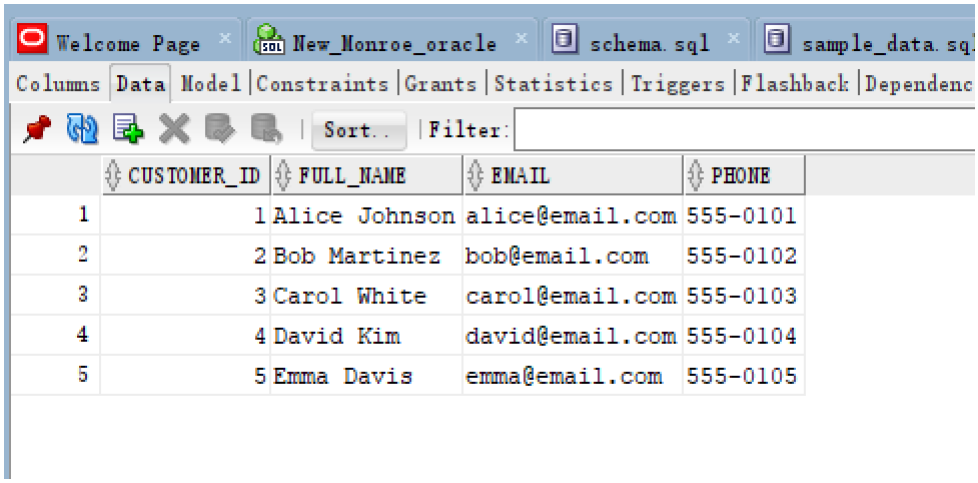
- **schema.sql**
  Contains all the CREATE TABLE statements and constraints
- **sample_data.sql**
  Contains sample data inserted into each table for testing
- **reset.sql**
  Drops all tables so the database can be reset easily

**Screenshot**

1. **ER diagram**: diagram showing the main entities in the system and how they are related, including customer, restaurant, orders, and menu_item and order_item.




2. **Customer table**: CUSTOMER table storing basic customer information such as name, email, and phone number. Each customer is uniquely identified by customer_id.




| | CUSTOMER_ID | FULL_NAME | EMAIL | PHONE |
|---|---|---|---|---|
| 1 | 1 | Alice Johnson | alice@email.com | 555-0101 |
| 2 | 2 | Bob Martinez | bob@email.com | 555-0102 |
| 3 | 3 | Carol White | carol@email.com | 555-0103 |
| 4 | 4 | David Kim | david@email.com | 555-0104 |
| 5 | 5 | Emma Davis | emma@email.com | 555-0105 |

3. **MENU_ITEM table**: MENU_ITEM table showing the items offered by each restaurant, including item name, price, and availability status

| | MENU_ITEM_ID | RESTAURANT_ID | ITEM_NAME | PRICE | IS_AVAILABLE |
|---|---|---|---|---|---|
| 1 | 1 | 1 | Classic Burger | 8.99 | YES |
| 2 | 2 | 1 | Cheese Burger | 9.99 | YES |
| 3 | 3 | 1 | French Fries | 3.49 | YES |
| 4 | 4 | 1 | Onion Rings | 3.99 | NO |
| 5 | 5 | 2 | Spaghetti Carbonara | 12.99 | YES |
| 6 | 6 | 2 | Penne Arrabbiata | 11.49 | YES |
| 7 | 7 | 2 | Garlic Bread | 3.99 | YES |
| 8 | 8 | 3 | Salmon Roll | 13.99 | YES |
| 9 | 9 | 3 | Tuna Sashimi | 15.99 | YES |
| 10 | 10 | 3 | Miso Soup | 2.99 | YES |

4. **ORDER_ITEM table**: ORDER_ITEM table linking orders and menu items, showing which items are included in each order along with quantity and price.

| | ORDER_ITEM_ID | ORDER_ID | MENU_ITEM_ID | QUANTITY | UNIT_PRICE |
|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 2 | 8.99 |
| 2 | 2 | 1 | 3 | 1 | 3.49 |
| 3 | 3 | 2 | 5 | 1 | 12.99 |
| 4 | 4 | 2 | 6 | 1 | 11.49 |
| 5 | 5 | 2 | 7 | 1 | 3.99 |
| 6 | 6 | 3 | 8 | 1 | 13.99 |
| 7 | 7 | 3 | 9 | 1 | 15.99 |
| 8 | 8 | 3 | 10 | 1 | 2.99 |
| 9 | 9 | 4 | 2 | 1 | 9.99 |
| 10 | 10 | 4 | 3 | 1 | 3.49 |
| 11 | 11 | 5 | 5 | 1 | 12.99 |
| 12 | 12 | 5 | 7 | 1 | 3.99 |

5. **ORDERS table**: ORDERS table storing: ORDERS table storing order details such as the customer, restaurant, order date, status, and total amount.

| | ORDER_ID | CUSTOMER_ID | RESTAURANT_ID | ORDER_DATE | ORDER_STATUS | TOTAL_AMOUNT |
|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 10-JAN-25 | DELIVERED | 21.47 |
| 2 | 2 | 2 | 2 | 11-JAN-25 | DELIVERED | 28.47 |
| 3 | 3 | 3 | 3 | 12-JAN-25 | PREPARING | 32.97 |
| 4 | 4 | 4 | 1 | 13-JAN-25 | PENDING | 13.48 |
| 5 | 5 | 5 | 2 | 14-JAN-25 | CONFIRMED | 16.98 |

6. **RESTAURANT table**: RESTAURANT table storing restaurant information, including restaurant name and location.

| | RESTAURANT_ID | RESTAURANT_NAME | LOCATION |
|---|---|---|---|
| 1 | 1 | The Burger House | 123 Main St, New York |
| 2 | 2 | Pasta Palace | 456 Oak Ave, New York |
| 3 | 3 | Sushi World | 789 Elm St, New York |